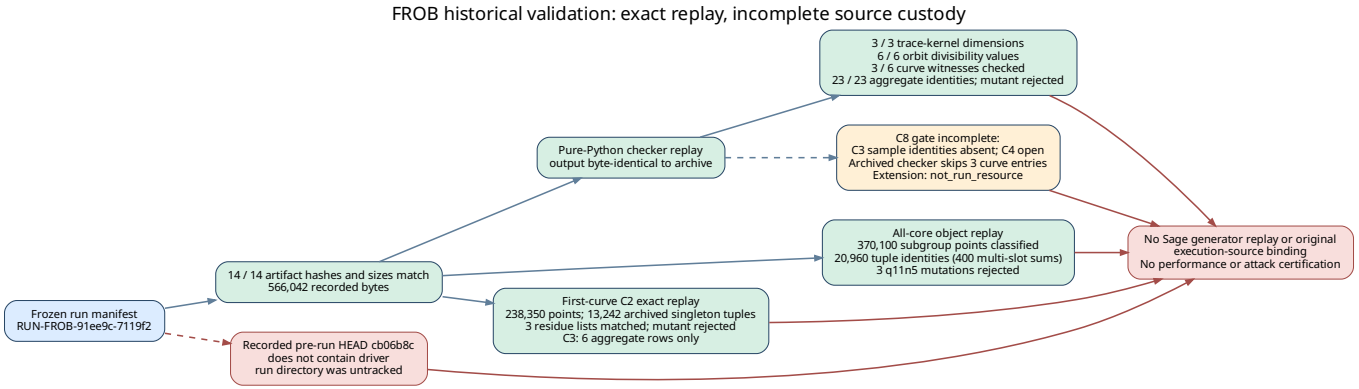
Historical FROB checker replay and source custody

Validation audit, 22 October 2026

Scope. This checks frozen run `RUN-FROB-91ee9c-7119f2` at source tree `e83bb2f14414c9669360816aeee5876dfcfd4384`. It is an integrity audit, not a new FROB measurement or a research verdict. The run directory was copied before executing its pure-Python checker; frozen bytes were not edited.

Check	Result	Limit
Archived artifact hashes and sizes	14/14 match; 566,042 bytes	Storage integrity only
Checker output	Byte-identical replay	Same archived checker source
Trace-kernel dimension; orbit divisibility	3/3; 6/6	Recorded core cells
Curve point/order/Frobenius witnesses	3/6 checked	Other three skipped for size
Aggregate rational identities	23/23; mutant rejected	Counts treated as inputs
New all-core object-arm replay	370,100 points; 20,960 tuples	400 multi-slot sums; 3 q11 mutants
First-curve C2 replay	238,350 points; 13,242 tuples	3 exact residue lists; singleton only
C3 aggregate replay	6 rows; 26,620 generated tuples	Archived sampled lists absent
Remaining C8 work	Open	C3 identities and C4 null curves
Extension cell	`not_run_resource`	540-second historical checkpoint



Source custody. The manifest's pre-run HEAD `cb06b8c20c2d6559041c45c4a4b29dd7f132fdf5` lacks the driver; the run directory was untracked. Hash-pinned command and environment files do not bind that HEAD to the executed source.

Boundary. The archived checker's three `ok` curve checks recompute no tuple sums. The new replays cover all object arms and the first-curve C2 lists; C3 is aggregate-only and C4 remains open. C8 and the success gate are unestablished. Recover C3 sample identities, re-enumerate C4, and resolve source custody before any original-run verdict. Receipts: `audit.json`, `core-object-replay.json`, `c2-c3-control-replay.json`.